

SESIÓN 4 – JUEGO ONLINE

Validación, permisos y seguridad

La Sesión 4 tiene tres ideas principales: evitar datos inválidos, controlar qué puede hacer cada usuario y evitar la inyección SQL.

1. Evitar datos inválidos con CHECK

Se agregan reglas a la tabla Jugadores:

```
ALTER TABLE Jugadores
  ADD CONSTRAINT chk_monedas_no_negativas CHECK (monedas >= 0),
  ADD CONSTRAINT chk_experiencia_no_negativa CHECK (experiencia >= 0),
  ADD CONSTRAINT chk_nivel_minimo CHECK (nivel >= 1);
```

- monedas no puede ser menor que 0.
- experiencia no puede ser menor que 0.
- nivel no puede ser menor que 1.

```
UPDATE Jugadores SET monedas = -100 WHERE jugador_id = 1;
```

✗ La base de datos rechaza el cambio porque rompe la regla CHECK.

También se pueden validar las partidas:

```
CHECK (puntaje >= 0)
CHECK (resultado IN ('victoria', 'derrota', 'jugada'))
```

Así se evita guardar valores inválidos, por ejemplo un resultado como 'tramposo'.

Idea principal: la propia base de datos protege la información aunque alguien intente introducir datos incorrectos.

2. Roles y permisos

Los roles permiten que cada tipo de usuario tenga diferentes permisos. Es como tener rangos dentro del juego.

Jugador

```
CREATE ROLE rol_jugador;

GRANT EXECUTE ON PROCEDURE juego_online.comprar_item_unico TO rol_jugador;
```

El jugador puede consultar información y ejecutar procedimientos permitidos, pero no puede modificar directamente sus monedas o nivel.

Moderador

```
CREATE ROLE rol_moderador;  
GRANT rol_jugador TO rol_moderador;
```

El moderador hereda los permisos del jugador.

```
GRANT UPDATE (baneado) ON juego_online.Jugadores TO rol_moderador;
```

Además, puede cambiar el estado baneado de los jugadores, pero no sus monedas.

Administrador

```
CREATE ROLE rol_administrador;  
GRANT rol_moderador TO rol_administrador;
```

El administrador hereda los permisos del moderador y del jugador.

```
GRANT UPDATE (monedas, nivel, experiencia)  
ON juego_online.Jugadores TO rol_administrador;  
  
GRANT DELETE ON juego_online.Jugadores TO rol_administrador;
```

También puede modificar monedas, nivel y experiencia, y eliminar jugadores.

Orden de permisos: JUGADOR → MODERADOR → ADMINISTRADOR. Cada nivel tiene más permisos.

3. Evitar la inyección SQL

La inyección SQL ocurre cuando una entrada del usuario puede interpretarse como código SQL. Una aplicación vulnerable podría construir una consulta mezclando directamente el texto ingresado.

```
"SELECT * FROM Jugadores WHERE nombre = " + texto_usuario + ""
```

Para reducir este riesgo, se puede utilizar un procedimiento con un parámetro. El texto recibido se trata como un dato.

```
CREATE PROCEDURE buscar_jugador_seguro(  
    IN p_nombre_usuario VARCHAR(50)  
)  
  
    SELECT jugador_id, nombre_usuario, nivel, monedas  
    FROM Jugadores  
    WHERE nombre_usuario = p_nombre_usuario;  
  
CALL buscar_jugador_seguro('DragonSlayer99');
```

Busca normalmente al jugador indicado.

Si alguien intenta ingresar:

```
' OR '1'='1
```

el valor se interpreta como un nombre de usuario, no como una condición SQL que deba ejecutarse.

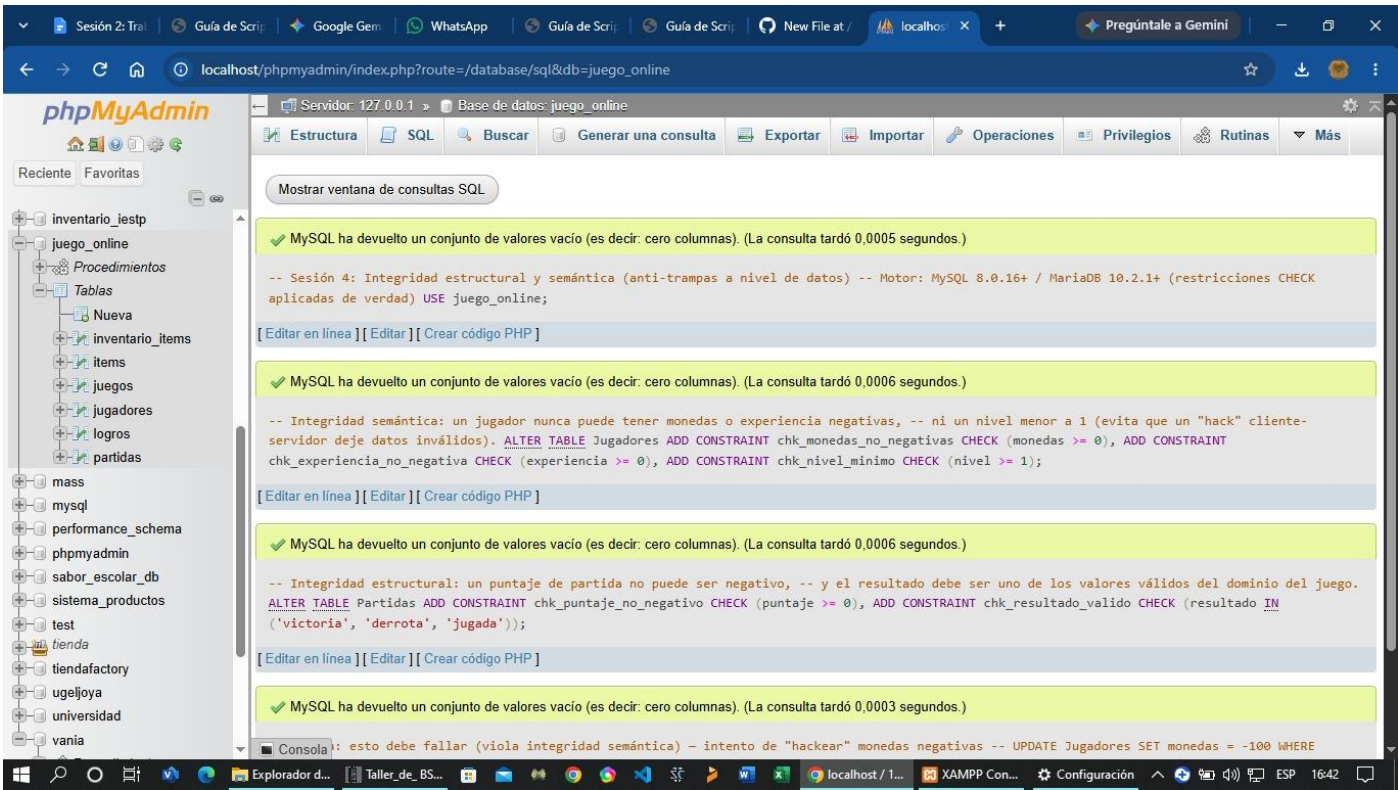
Idea principal: los datos que proporciona el usuario deben tratarse como datos y no como código SQL.

4. Lo que debes memorizar de la Sesión 4

Concepto	¿Para qué sirve?
CHECK	Protege los datos y evita valores inválidos.
GRANT + ROLE	Controla los permisos de cada tipo de usuario.
Parámetros en procedimientos	Ayudan a evitar que una entrada del usuario se interprete como código SQL.

5. ¿Qué hace la Sesión 4?

Garantiza que los datos sean válidos, controla los permisos de cada tipo de usuario y ayuda a evitar que una entrada maliciosa pueda ejecutar SQL no autorizado.



Sesión 2: Tra... Guía de Scrip... Google Gem... WhatsApp... Guía de Scrip... Guía de Scrip... New File at / localhost: x + Pregúntale a Gemini

localhost/phpmyadmin/index.php?route=/table/sql&db=juego_online&table=j

phpMyAdmin

Reciente Favoritas

- inventario_iesp
- juego_online
 - Procedimientos
 - Tablas
 - Nueva
 - inventario_items
 - items
 - juegos
 - jugadores
 - logros
 - partidas
- mass
- mysql
- performance_schema
- phpmyadmin
- sabor_escolar_db
- sistema_productos
- test
- tienda
- tiendafactory
- ugeljoya
- universidad
- vania

Servidor: 127.0.0.1 » Base de datos: juego_online

Estructura SQL Buscar Generar una consulta Exportar Importar Operaciones Privilegios Rutinas Más

[Editar en línea] [Editar] [Crear código PHP]

⚠ La selección actual no contiene una columna única. La edición de la grilla y los enlaces de copiado, eliminación y edición no están disponibles.

✓ Mostrando filas 0 - 0 (total de 1, La consulta tardó 0.0005 segundos.)

-- Resultado esperado: 0 filas (porque ningún jugador se llama literalmente ' OR '1'='1 ') [CALL](#) buscar_jugador_seguro('DragonSlayer99');

[Editar en línea] [Editar] [Crear código PHP]

☐ Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla

Opciones extra

jugador_id	nombre_usuario	nivel	monedas
1	DragonSlayer99	12	1500

☐ Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla

Operaciones sobre los resultados de la consulta

[Imprimir](#) [Copiar al portapapeles](#) [Crear vista](#)

[Guardar esta consulta en favoritos](#)

☐ Permitir que todo usuario pueda acceder a este favorito

Consola

Explorador d... Taller de BS... localhost / 1... XAMPP Con... Configuración 16:46